FLAPPYBIRD RELOADED

PROJEKT P2 SOSE 2016 – TIM MANDELBAUM 2053403

INHALTSVERZEICHNIS

Inhalt

Dokumentation _____ 1

Dokumentation

WAS HABE ICH GEMACHT

Für das freiwillige Projekt im Kurs Programmieren 2 (SoSe 2016), habe ich die im Semester entstandenen Adaption des Spiels „Flappy Bird“ um folgende Funktionen erweitert:

- Spieler und Level: der Spieler kann sich jetzt mit seinem Namen anmelden und aus 3 verschiedenen Flappy Birds wählen.
- Die jeweiligen Flappy Birds fliegen durch unterschiedliche Welten. Aktuell gibt es die gewohnte grüne Oberflächenwelt, eine Winterwelt und eine Lavaunterwelt.
- Highscore: ich habe das Spiel um die Funktion erweitert, seine erreichte Highscore zu speichern. Der erreichte Punktestand wird mit einem Client an einen Echoserver geschickt, dieser speichert alle Punktestände in einer Datei ab und liefert diese dann sortiert an das Spiel zurück.
- Feuer: Flappy Bird kann jetzt auch Feuer spucken! Jeder Feuerball kann Hindernisse zerstören, zieht dem Spieler aber auch viele Punkte seiner Highscore ab. Man sollte diese Feuerbälle also nur im Notfall benutzen.

WIE BEDIENT MAN ES

Das Spiel lässt sich ganz einfach mit der Tastatur oder Maus lenken. Flappbird kann entweder durch Klicken der linken Maustaste oder mit Hilfe der Pfeiltaste (oben) zum Steigen gebracht werden.

Flappy Bird sinkt auf seiner Flugbahn automatisch wieder ab, trifft man ein Hindernis oder kollidiert mit dem oberen oder unteren Rand des Bildschirms, endet das Spiel.

Mit der Leertaste kann Flappybird jetzt auch Feuerbälle abfeuern, diese können Hindernisse zerstören. Jeder abgegebene Feuerball zieht jedoch Punkte ab und bringt den Spieler schnell in einen negativen Punktestand.

Es geht also darum geschickt durch die Hindernisse zu fliegen und so viele Punkte wie möglich zu sammeln.

Den Echoserver für die Highscore muss man aktuell noch manuell starten, bevor man die FlappyExe.java ausführt. Dieser würde eigentlich ja eh permanent laufen und immer nur entsprechend auf Anfrage reagieren.

WAS HAT NICHT GEKLAPPT

Ich hatte geplant das Spiel durch die Variante eines fliegenden Gegners zu erweitern, das stellte sich aber als nicht wirklich gut spielbar heraus.

Die Öffnungen in den Rohren erscheinen nach einem Zufallsablauf, kreuzten jetzt noch Gegner die Flugbahn, war es eigentlich unmöglich das Spiel zu spielen. Deshalb habe ich mich entschieden, diese wieder aus dem Spiel zu entfernen.

<<Java Class>>
EchoServer
(default package)
inputStream: ObjectInputStream
outputStream: ObjectOutputStream
clientSocket: Socket
port: int
EchoServer()
closeConnection():void
main(String[]):void
<<Java Class>>
FlappyExe
(default package)
FlappyExe()
initUI():void
main(String[]):void
<<Java Class>>
Fireball
(default package)
fireballSpeed: int
boardWidth: int
imgSrc: String
Fireball(int,int)
initFireball():void
move():void
-fireballs
0..*
<<Java Class>>
Sprite
(default package)
x: int
y: int
width: int
height: int
visible: boolean
image: Image
Sprite(int,int)
Sprite(int,int,int,int)
getImageDimensions():void
loadImage(String):void
getImage():Image
getX():int
getY():int
getWidth():int
getHeight():int
isVisible():boolean
setVisible(Boolean):void
getBounds():Rectangle
<<Java Class>>
EchoClient
(default package)
outputStream: ObjectOutputStream
inputStream: ObjectInputStream
socket: Socket
port: int
host: String
EchoClient()
updateHighscore(Player):Highscore
closeConnection():void
<<Java Class>>
Saeulen
(default package)
directionX: int
imgSrc: String
Saeulen(int,int,int,int,int)
initSaeulen():void
move():void
setImgSrc(int):void
0..*
#saeulenElemente
<<Java Class>>
Flappy
(default package)
directionY: int
fallVelocity: int
climbRate: int
imgSrc: String
Flappy(int,int)
initFlappy():void
move():void
fall():void
getFireballs():ArrayList
keyPressed(keyEvent):void
keyReleased(keyEvent):void
mouseClicked(MouseEvent):void
mouseReleased(MouseEvent):void
tire():void
setImgSrc(int):void
-flappy
0..1
<<Java Class>>
Highscore
(default package)
directory: String
Highscore()
getHighscore():ArrayList<Player>
getHighscoreAsString():String
setHighscore(String,int):void
printHighScore():String
-highscore
0..1
-highscore
0..1
<<Java Class>>
GamePanel
(default package)
timer: Timer
ingame: boolean
flappyX: int
flappyY: int
saeulenGap: int
saeulenWidth: int
saeulenType: int
gamePanelHeight: int
DELAY: int
counter: int
counterMatch: int
score: int
fireballMalus: int
initDone: boolean
once: boolean
GamePanel()
initGamePanel():void
startTimer():void
initSaeulen():void
actionPerformed(actionEvent):void
inGame():void
updateFlappy():void
updateSaeulen():void
updateFireballs():void
updateBackground():void
checkCollision():void
drawObjects(Graphics):void
drawGameOver(Graphics):void
paintComponent(Graphics):void
drawString(Graphics,String,int,int):void
<<Java Class>>
Background
(default package)
directionX: int
imgSrc: String
Background(int,int)
initBackground():void
move():void
setImgSrc(int):void
-bg1
0..1
-bg2
0..1
<<Java Class>>
SaeulenDouble
(default package)
y: int
SaeulenDouble(int,int,int,int,int)
*saeulen
0..*
<<Java Class>>
Player
(default package)
name: String
score: int
Player(String,int)
getName():String
getScore():int
setScore(int):void
setName(String):void
-player
0..1
<<Java Class>>
keyTranslateAdapter
(default package)
keyTranslateAdapter()
keyReleased(keyEvent):void
keyPressed(keyEvent):void
<<Java Class>>
mouseTranslateAdapter
(default package)
mouseTranslateAdapter()
mouseClicked(MouseEvent):void
mouseReleased(MouseEvent):void